

Quipster: 面向大学校园的兴趣社交平台设计与实现

Project Proposal

魏同烜, 黎高和, 陈百川, 王宜轩, 王瑞麟 *

March 22, 2026

Abstract

本文提出一个面向大学校园环境的社交平台 Quipster。系统通过校园邮箱认证确保用户真实性，并利用兴趣标签与推荐机制提升社交匹配效率。平台同时提供即时通信、动态互动、校园问答、匿名树洞以及学习搭子等功能，并通过社区信用体系维护平台秩序。本文从系统需求、系统架构、技术栈以及人工智能辅助功能等方面对系统设计进行初步规划。

Contents

1	项目背景	3
1.1	互联网社交的发展	3
1.2	校园社交需求	3
1.3	项目定位	3
2	项目范围与需求分析	3
2.1	项目范围	3
2.2	功能需求	4
2.3	非功能需求	4
3	项目功能模块设计	5
3.1	账号与安全认证模块	5
3.2	个人资料与兴趣管理模块	6
3.3	核心交友与匹配模块	6
3.4	社交互动模块	7
3.5	社区管理与信用模块	7

*学号: 2353902, 2351097, 2351839, 2352494, 2353118.

3.6	匿名树洞与情绪交流模块	7
3.7	校园问答与经验分享模块	8
3.8	学习搭子与任务匹配模块	8
4	系统逻辑架构与物理架构设计	9
4.1	系统逻辑架构	9
4.1.1	表示层 (Presentation Layer)	9
4.1.2	业务逻辑层 (Application / Service Layer)	10
4.1.3	数据层 (Data Layer)	10
4.2	系统物理部署架构	11
4.2.1	前端服务器 (Web Server)	11
4.2.2	应用服务器 (Application Server)	11
4.2.3	数据库服务器 (Database Server)	12
4.2.4	缓存服务器 (Cache Server)	12
5	候选开发技术栈	12
6	初步技术框架	13
7	项目所用编程语言	13
8	项目管理与系统测试方案	14
9	人工智能解决方案设想	14
9.1	AI 问答助手	15
9.2	AI 情绪识别	15
9.3	AI 聊天破冰助手	15
9.4	AI 智能内容识别	15
10	风险分析与未来扩展	16
10.1	技术风险	16
10.2	项目管理风险	16
10.3	未来功能扩展	17
11	项目日程详细安排	17

1 项目背景

1.1 互联网社交的发展

随着移动互联网和智能终端的普及，线上社交平台已经成为年轻人日常交流的重要方式。主流社交平台（如微信、QQ 等）主要以熟人社交为核心，其社交关系往往建立在已有现实关系基础之上，对于希望拓展新社交圈或寻找兴趣相投伙伴的用户而言，匹配效率和社交体验仍然存在一定局限。

1.2 校园社交需求

大学校园具有相对独立的社交环境，学生群体在学习、兴趣发展以及社团活动中具有丰富的社交需求。然而在实际环境中，学生社交仍然存在一些问题：

- 不同学院和专业之间交流机会有限，跨专业社交渠道较少；
- 传统社交方式主要依赖课程或线下活动，寻找兴趣相同伙伴效率较低；
- 大型开放社交平台用户来源复杂，缺乏校园身份认证机制；
- 用户隐私保护和信息安全问题仍然受到广泛关注。

1.3 项目定位

针对上述问题，本项目设计并实现一个面向大学校园环境的社交平台——Quipster。平台以校园用户为核心，通过校园邮箱认证机制确保用户身份真实性，并通过兴趣标签与智能推荐算法提高用户匹配效率。同时，系统将引入隐私分级控制和用户信用机制，以构建安全、可信的校园社交环境。

总体而言，Quipster 旨在打造一个融合兴趣社交、信息交流与校园活动互动的综合平台，为大学生提供更加高效、可靠的校园社交体验。

2 项目范围与需求分析

2.1 项目范围

本项目旨在设计并实现一个面向大学校园环境的在线社交平台 Quipster。系统主要服务对象为同济大学在校学生，通过校园身份认证机制构建可信用户环境。平台支持用户基于兴趣标签进行社交匹配，并提供信息发布、即时交流和社群活动组织等功能，从而提升校园内部的社交效率和互动体验。

系统主要包含用户管理、兴趣匹配、即时通信、内容互动以及社区治理等核心模块。

2.2 功能需求

根据系统目标与使用场景，平台需要实现以下主要功能需求：

- **FR1 用户身份认证**：系统应支持通过校园邮箱进行注册与身份验证，以确保用户为真实校园成员。
- **FR2 用户资料管理**：用户能够创建和管理个人资料，包括基本信息、兴趣标签以及个人介绍等内容。
- **FR3 兴趣标签管理**：系统应允许用户选择或自定义兴趣标签，并基于标签建立用户兴趣模型。
- **FR4 智能匹配推荐**：系统应根据用户兴趣标签、行为数据等信息，通过推荐算法向用户推荐潜在好友或兴趣相似用户。
- **FR5 即时聊天功能**：平台应提供用户之间的实时聊天功能，支持文本消息发送与接收。
- **FR6 动态发布与互动**：用户可以发布动态内容，并支持点赞、评论等互动行为。
- **FR7 兴趣社群与活动**：系统应支持创建兴趣社群，并允许用户发起或参与校园活动。
- **FR8 社区信用体系**：平台应建立用户信用评价机制，用于约束不良行为并维护社区秩序。
- **FR9 匿名树洞**：用户可以匿名发布内容，分享自己的学习压力、生活困惑或情感问题，从而获得其他用户的建议与支持。
- **FR10 校园问答**：该模块为用户提供一个开放的讨论空间，使用户能够提出问题、分享经验，并通过互动交流获取有价值的信息。
- **FR11 学习搭子**：用户可以通过该功能快速找到具有相同目标的学习伙伴，提高学习效率并增强学习互动。

2.3 非功能需求

除功能需求外，系统还需满足以下关键非功能需求：

- **NFR1 系统安全性**：系统应采用安全的身份认证和数据传输机制，以防止未授权访问和数据泄露。
- **NFR2 隐私保护**：用户的个人信息和聊天数据应受到保护，并提供合理的隐私控制机制。

- **NFR3 系统性能：**系统应能够支持多用户同时在线，并保持稳定的响应速度。
- **NFR4 可扩展性：**系统架构应具备良好的可扩展性，以支持未来功能扩展和用户规模增长。

3 项目功能模块设计

根据系统需求分析，Quipster 平台整体功能可以划分为五个核心模块：账号与安全认证模块、个人资料与兴趣管理模块、核心交友与匹配模块、社交互动模块以及社区管理与信用模块。各模块之间相互协作，共同构成完整的校园社交系统架构。

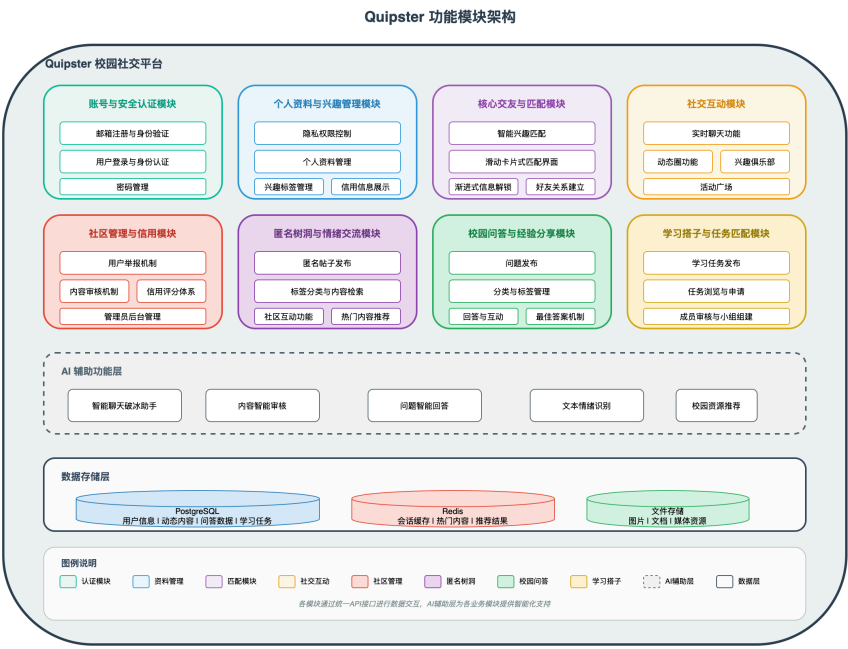


Figure 1: 系统模块设计图

3.1 账号与安全认证模块

账号与安全认证模块主要负责用户身份验证与账户安全管理，是系统安全体系的重要组成部分。为了保证平台用户的真实性与可信度，系统采用同济大学校园邮箱进行注册认证，从源头限制非校园用户进入平台。

该模块主要包括以下功能：

- **邮箱注册与身份验证：**用户需通过同济大学校园邮箱完成注册，并通过邮件验证确认身份。
- **用户登录与身份认证：**系统支持账号密码登录，并通过安全认证机制保证账户访问安全。

- 密码管理：提供密码找回与密码重置功能，确保用户在遗忘密码时能够安全恢复账户访问权限。

3.2 个人资料与兴趣管理模块

个人资料与兴趣管理模块主要用于维护用户的个人信息和兴趣数据，是实现用户匹配与推荐的重要数据基础。通过建立结构化的用户信息模型，系统能够更加准确地分析用户特征并进行兴趣匹配。

该模块主要包含以下功能：

- 隐私权限控制：用户可以对个人资料、动态内容以及社交关系设置不同的可见范围，实现分级隐私管理。
- 个人资料管理：用户可以编辑和更新个人基本信息，包括专业、年级以及个人简介等内容。
- 兴趣标签管理：用户可以从系统提供的兴趣标签库中选择标签，也可以创建自定义标签，用于描述个人兴趣与爱好。
- 兴趣与学术领域展示：系统支持展示用户的研究方向或兴趣领域，方便用户寻找志同道合的伙伴。
- 信用信息展示：平台通过信用评分体系记录用户行为表现，用户可以查看自己的信用等级与历史变化情况。

3.3 核心交友与匹配模块

核心交友与匹配模块是系统的关键功能模块，其主要目标是通过数据分析和推荐算法提高用户社交匹配效率。系统基于用户兴趣标签、专业背景以及行为数据等多维度信息，对用户之间的相似度进行计算，并生成潜在好友推荐列表。

该模块主要功能包括：

- 智能兴趣匹配：基于多维度标签（兴趣、专业、活动参与记录等）计算用户之间的相似度，并生成推荐结果。
- 滑动卡片式匹配界面：系统采用类似滑动卡片的交互方式展示推荐用户，从而提升匹配过程的直观性与趣味性。
- 渐进式信息解锁机制：在用户互动过程中逐步解锁更多个人信息，以平衡社交体验与隐私保护。
- 好友关系建立：当双方匹配成功后，系统建立好友关系并允许进一步交流互动。

3.4 社交互动模块

社交互动模块为用户提供多样化的互动方式，是促进用户交流和增强平台活跃度的重要组成部分。该模块通过实时通信和内容分享机制，构建活跃的校园社交环境。

主要功能包括：

- 实时聊天功能：支持用户之间的一对一即时聊天，并支持文本、图片和表情等多种消息类型。
- 动态圈功能：用户可以发布图文动态，并支持浏览、点赞和评论等互动行为。
- 兴趣俱乐部：用户可以创建或加入基于共同兴趣的社群，实现主题交流。
- 活动广场：平台支持发布校园活动信息，用户可以浏览、报名并参与相关活动。

3.5 社区管理与信用模块

社区管理与信用模块用于维护平台的良好社区秩序，通过行为监管和信用评价机制减少不良行为的发生。该模块同时为平台管理员提供管理工具，以保证社区环境的健康发展。

主要功能包括：

- 用户举报机制：用户可以对违规内容或不良行为进行举报。
- 内容审核机制：系统对动态内容进行审核，以防止不当信息传播。
- 信用评分体系：根据用户行为记录动态调整用户信用分，用于评价用户信誉。
- 管理员后台管理：管理员可以处理举报、审核内容，并对违规用户进行警告、禁言或封禁等操作。

3.6 匿名树洞与情绪交流模块

匿名树洞与情绪交流模块旨在丰富平台的社交形式，并满足大学生在情绪表达与心理交流方面的需求。该模块允许用户在不暴露真实身份的情况下发布内容，分享学习压力、生活困惑或情感问题，从而获得其他用户的建议与支持。通过匿名机制，用户能够更加自由地表达真实想法，增强平台的情感交流属性，并进一步提升平台的社区互动氛围。

主要功能包括：

- 匿名帖子发布：用户可以发布匿名帖子，帖子内容包括标题、正文以及相关标签。
- 标签分类与内容检索：系统支持通过标签对帖子进行分类管理，提高信息检索效率。
- 社区互动功能：用户可以对帖子进行点赞、评论或收藏，从而形成社区讨论氛围。

- 热门内容推荐：系统根据互动情况（点赞数、评论数和浏览量）生成热门帖子。
- 情绪识别与支持推荐：结合自然语言处理技术识别用户情绪，并推荐相关校园资源或支持信息。

3.7 校园问答与经验分享模块

校园问答模块旨在促进校园内部的知识交流与经验共享，为用户提供开放的讨论空间，使学生能够提出问题、分享经验并获取有价值的信息。

主要功能包括：

- 问题发布：用户可以发布课程学习、考研经验、实习信息或校园生活相关问题。
- 分类与标签管理：平台支持问题分类与标签机制，以提高信息组织与检索效率。
- 回答与互动：用户可以对问题进行回答，并通过点赞或评论进行互动。
- 最佳答案机制：提问者可以从回答中选择最有帮助的一条作为最佳答案。
- 社区激励机制：优质回答者可以获得积分或社区声誉奖励。

系统还可以通过检索资料库生成智能回答，辅助用户解决问题。

3.8 学习搭子与任务匹配模块

学习搭子与任务匹配模块旨在帮助学生在在学习过程中快速找到合适的学习伙伴，从而提高学习效率并增强学习互动。

主要功能包括：

- 学习任务发布：用户可以发布学习需求，例如课程复习、口语练习或考研学习小组。
- 任务浏览与申请：其他用户可以浏览任务列表并申请加入相应学习任务。
- 成员审核与小组组建：任务发布者可以审核申请成员，并在人数满足条件后创建学习小组。
- 学习小组交流空间：系统为学习小组提供讨论空间或群聊，用于交流和资料分享。

通过学习搭子与任务匹配模块，平台不仅能够提升学习协作效率，还能够促进学生之间形成积极的学习型社交关系。

4 系统逻辑架构与物理架构设计

为了保证系统具有良好的可扩展性、可维护性以及稳定的性能表现，Quipster 平台采用前后端分离的多层架构设计。通过将系统功能划分为不同层次，各层之间通过清晰的接口进行交互，从而降低系统耦合度并提高系统扩展能力。

系统整体从两个层面进行设计：系统逻辑架构与系统物理部署架构。

4.1 系统逻辑架构

在逻辑层面上，系统采用典型的分层架构模式，将系统划分为表示层（Presentation Layer）、业务逻辑层（Application / Service Layer）和数据层（Data Layer）三个主要层次。各层之间通过标准 API 接口进行通信，从而实现模块化设计并提高系统的可扩展性。

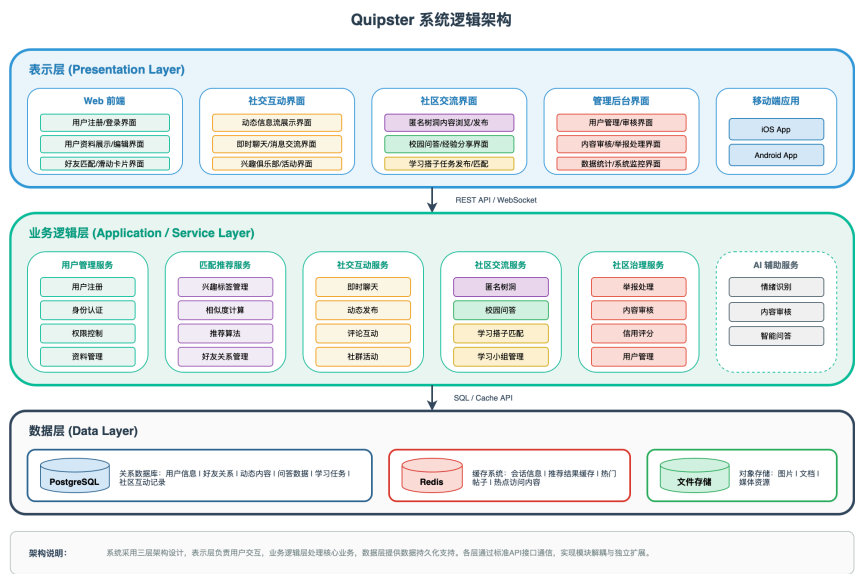


Figure 2: 系统逻辑架构图

4.1.1 表示层（Presentation Layer）

表示层主要负责用户界面展示以及用户交互逻辑。该层通常由 Web 前端或移动端应用构成，负责接收用户输入并将请求发送到后端服务，同时将后端返回的数据以可视化形式呈现给用户。

在 Quipster 系统中，表示层主要提供以下功能界面：

- 用户注册与登录界面
- 用户资料展示与编辑界面
- 好友匹配与滑动卡片界面
- 动态信息流展示界面

- 匿名树洞内容浏览与发布界面
- 校园问答与经验分享界面
- 学习搭子任务发布与匹配界面
- 即时聊天与消息交流界面

通过统一的前端界面设计，用户可以在同一平台上完成社交匹配、信息交流以及学习协作等多种活动。

4.1.2 业务逻辑层 (Application / Service Layer)

业务逻辑层是系统的核心处理层，主要负责处理用户请求并实现系统业务规则。该层通过 API 接口接收来自前端的请求，并调用相关服务模块完成业务处理。

在 Quipster 系统中，主要业务模块包括：

- 用户管理服务（注册、登录、权限控制）
- 兴趣标签管理服务
- 匹配推荐服务（基于兴趣与行为数据）
- 社交互动服务（聊天、动态、评论）
- 匿名树洞服务
- 校园问答服务
- 学习任务匹配服务
- 社区管理服务（举报、审核、信用评分）

此外，系统还可以引入人工智能辅助模块，用于实现智能聊天破冰助手、内容智能审核、问题智能回答以及文本情绪识别等功能。通过模块化服务设计，系统能够根据需求灵活扩展新的功能模块，并提升整体系统的可维护性。

4.1.3 数据层 (Data Layer)

数据层负责系统数据的持久化存储和管理，为上层业务提供稳定的数据支持。该层主要包括关系数据库和缓存系统两部分。

在 Quipster 系统中：

- PostgreSQL 数据库用于存储用户信息、好友关系、动态内容、问答数据、学习任务以及社区互动记录等核心业务数据。

- Redis 缓存系统用于存储会话信息、推荐结果缓存、热门帖子数据以及热点访问内容，以提升系统访问效率并减少数据库访问压力。

通过数据库与缓存系统的结合，系统能够在保证数据可靠性的同时提高访问性能。

通过上述分层设计，系统可以实现良好的模块解耦，使前端界面、业务逻辑以及数据存储能够独立演进和扩展，从而提高系统整体的可维护性与扩展能力。

4.2 系统物理部署架构

在物理部署层面，系统采用典型的多服务器部署结构，通过不同服务器节点承担不同系统职责，从而提高系统整体性能和稳定性。

系统主要由以下几个核心服务器组成。

4.2.1 前端服务器 (Web Server)

前端服务器主要负责部署前端应用程序，并向用户提供 Web 访问入口。该服务器通常部署静态资源 (HTML、CSS、JavaScript 等)，并通过 HTTP 或 HTTPS 协议向浏览器提供访问服务。

在实际部署中，前端服务器还可以通过反向代理与负载均衡机制，提高系统访问性能并增强系统稳定性。

4.2.2 应用服务器 (Application Server)

应用服务器负责运行后端业务逻辑，是系统的核心计算节点。该服务器主要负责处理用户请求、执行业务逻辑、调用数据库服务，并向前端返回处理结果。

在 Quipster 系统中，应用服务器主要承担以下任务：

- 用户认证与会话管理
- 好友匹配算法计算
- 即时聊天消息处理
- 动态内容发布与管理
- 匿名树洞内容管理
- 校园问答与知识分享服务
- 学习任务与学习搭子匹配
- 社区审核与信用评分

此外，应用服务器还可以调用 AI 服务模块，实现智能推荐、情绪识别以及内容审核等辅助功能。

4.2.3 数据库服务器 (Database Server)

数据库服务器负责存储系统核心业务数据，并提供高可靠的数据访问服务。系统采用 PostgreSQL 关系数据库，用于存储用户资料、兴趣标签、好友关系、聊天记录、动态内容、问答数据以及学习任务信息等数据。

4.2.4 缓存服务器 (Cache Server)

缓存服务器采用 Redis 实现，用于缓存高频访问数据，例如用户会话信息、热门动态内容、热门问答问题以及用户推荐结果等。通过缓存机制可以减少数据库访问压力，从而提高系统响应速度并提升整体用户体验。

在系统运行过程中，用户请求首先到达前端服务器，随后通过 API 接口发送至应用服务器进行业务处理。应用服务器根据业务需求访问数据库服务器或缓存服务器，最终将处理结果返回给用户终端。

通过上述物理部署架构，系统能够在保证稳定运行的同时具备良好的扩展能力，以支持未来用户规模增长以及功能模块扩展。

5 候选开发技术栈

根据系统功能需求与系统架构设计，初步选择如下候选技术栈。各技术组件及其主要用途如表 1 所示。

Table 1: Quipster 系统候选技术栈

技术层级	技术组件	具体技术
Frontend	前端框架	React.js
Frontend	页面结构	HTML5
Frontend	页面样式	CSS3 / Flexbox / Grid
Frontend	前端逻辑	JavaScript (ES6+)
Frontend	网络请求	Axios / Fetch API
Frontend	实时通信	WebSocket
Backend	运行环境	Node.js
Backend	Web 框架	Express.js
Backend	身份认证	JWT
Backend	实时通信	Socket.io
Data Layer	关系数据库	PostgreSQL
Data Layer	缓存数据库	Redis
Infrastructure	Web 服务器	Nginx
Infrastructure	操作系统	Linux (Ubuntu)
Infrastructure	容器化	Docker (可选)
Infrastructure	版本管理	Git / GitHub

6 初步技术框架

为了提升系统的可扩展性与可维护性，Quipster 平台在系统设计阶段初步采用微服务架构（Microservice Architecture）。在该架构模式下，系统被划分为多个相互独立的服务模块，每个服务模块专注于特定的业务功能，并可以独立开发、部署与扩展。这种架构能够有效降低系统模块之间的耦合度，并提高系统整体的灵活性与可维护性。

在微服务架构下，各服务模块通过标准化接口进行通信，从而保证系统结构的清晰性和扩展能力。

系统的核心服务模块主要包括：

- 用户认证服务（Authentication Service）：负责用户注册、登录验证、会话管理以及权限控制。
- 匹配推荐服务（Matching & Recommendation Service）：基于兴趣标签与用户行为数据生成好友推荐列表，并支持学习搭子匹配。
- 即时聊天服务（Messaging Service）：负责实时消息通信、消息存储以及用户在线状态管理。
- 动态内容服务（Feed Service）：管理用户发布的动态信息以及相关互动行为。
- 社区内容服务（Community Service）：支持问答模块与匿名树洞模块的内容管理。
- 学习协作服务（Study Collaboration Service）：支持学习任务发布、学习小组创建以及成员匹配。
- 社区审核服务（Moderation Service）：负责举报处理、内容审核以及用户信用评分管理。

各服务模块之间通过 RESTful API 进行通信，并遵循统一的接口规范进行数据交换，从而保证系统的兼容性与扩展能力。

当系统规模进一步扩大时，系统架构可以扩展为 API Gateway 模式。在该模式下，所有外部请求首先通过 API Gateway 进入系统，由网关统一进行请求路由、身份认证以及流量控制，从而提高系统安全性并简化客户端访问结构。

通过上述技术框架设计，Quipster 平台能够在保证系统稳定运行的同时，为未来功能扩展和系统升级提供良好的技术基础。

7 项目所用编程语言

根据系统的技术架构设计，Quipster 平台将采用多语言协同开发的方式。不同系统模块根据其功能需求选择最适合的编程语言进行实现，从而提高开发效率并增强系统的可维护性与扩展能力。

本项目计划使用的主要编程语言包括以下两种。

JavaScript

JavaScript 主要用于前端界面开发，实现用户界面交互逻辑以及动态页面功能。通过现代 JavaScript 技术，可以构建动态信息流、好友匹配卡片以及即时聊天界面等交互功能。此外，JavaScript 还可以与 WebSocket 技术结合，实现实时通信功能，从而支持系统中的即时聊天和实时通知机制。

TypeScript

TypeScript 是 JavaScript 的扩展语言，在 JavaScript 基础上增加了静态类型系统。通过引入类型约束机制，TypeScript 可以在开发阶段提前发现潜在错误，从而提高代码质量和系统稳定性。本项目将在前端开发以及部分后端服务开发中使用 TypeScript，以提高代码的可读性、可维护性以及系统可靠性。

8 项目管理与系统测试方案

在项目开发过程中，团队将采用敏捷开发（Agile Development）方法进行项目管理。项目开发周期被划分为多个迭代阶段，每个迭代周期约为两周。在每个迭代周期中完成需求实现、功能开发、系统测试以及阶段评审，从而逐步完善系统功能并持续优化用户体验。

系统测试工作将贯穿整个开发过程，并主要包括以下几个阶段。

- **单元测试（Unit Testing）** 对系统中的各个功能模块进行独立测试，确保单个模块逻辑正确并能够正常运行。
- **集成测试（Integration Testing）** 对多个模块之间的接口与交互流程进行测试，以验证系统各组件之间的协同工作能力。
- **系统测试（System Testing）** 在接近真实运行环境的条件下，对系统整体功能和性能进行全面测试，以验证系统是否满足设计需求。
- **安全测试（Security Testing）** 重点检查系统在身份认证、数据访问控制以及隐私保护方面的安全性，防止未经授权的数据访问或信息泄露。

在测试工具方面，计划使用 **Jest** 对前端以及部分后端模块进行单元测试，并使用 **Postman** 对系统 API 接口进行调试与接口测试。此外，在系统部署阶段还将进行基础性能测试，以评估系统在多用户访问情况下的稳定性和响应能力。

9 人工智能解决方案设想

为了进一步提升平台的智能化水平与用户体验，Quipster 系统计划在内容服务与社交互动方面引入人工智能技术。与传统社交平台不同，本系统并不依赖人工智能进行好友匹配，而是采用基于兴趣标签的传统匹配算法完成用户推荐。在此基础上，人工智能技术将主要用于信息辅助、情绪识别、内容安全管理以及社交互动增强等功能。

通过引入自然语言处理（Natural Language Processing, NLP）和大语言模型（Large Language Model, LLM）等技术，系统能够在保证实现难度可控的前提下，为用户提供更加智能化的交互体验。

9.1 AI 问答助手

在校园问答模块中，系统将引入 AI 问答助手（AI Question Answering Assistant），用于帮助用户更快速地获得参考信息。当用户发布问题后，AI 系统可以从平台资料库中检索相关信息，并自动生成参考回答。

该模块主要包括问题理解与关键词提取、知识库检索以及答案生成三个步骤。系统首先对问题进行文本分析并提取关键主题词，然后在校园知识资料库中检索相关信息，最后通过自然语言生成技术整理并生成参考回答。

在技术实现方面，该模块可以采用关键词检索结合大语言模型 API 的方式实现，从而在人工回答出现之前为用户提供初步参考信息。

9.2 AI 情绪识别

在匿名树洞模块中，用户可以分享个人情绪、压力或生活困扰。系统将引入 AI 情绪识别功能（Emotion Detection），对用户文本内容进行情绪分析，并将情绪划分为积极、中性、焦虑、压力以及悲伤等类型。

当系统检测到明显负面情绪时，可以触发辅助支持机制，例如推荐相关帖子、推荐校园活动或提供心理健康资源信息。该功能能够帮助平台更好地理解用户情绪状态，并为用户提供适当的支持。

在技术实现方面，可以采用情绪词典分析方法、关键词匹配机制或调用现有情绪分析 API。

9.3 AI 聊天破冰助手

在社交平台中，用户在刚完成好友匹配时往往会因为缺乏话题而难以开始交流。为此，系统计划设计 AI 聊天破冰助手（AI Conversation Icebreaker Assistant）。

当两个用户匹配成功后，系统可以根据双方的兴趣标签自动生成聊天建议，例如推荐聊天话题、兴趣互动建议或简单互动小游戏，从而帮助用户更自然地开始交流。

该功能可以通过调用大语言模型 API 或基于兴趣标签的规则模板生成实现。

9.4 AI 智能内容识别

为了维护平台社区环境并防止违规内容传播，系统计划引入 AI 智能内容识别机制（AI Content Moderation），用于检测用户发布内容中的潜在风险。

该模块可以对动态内容、评论信息、树洞帖子以及聊天信息进行检测，并实现敏感词检测、违规内容识别以及风险内容标记等功能。当系统检测到可能存在违规风险的内

容时，将自动进行标记并提交管理员进行进一步审核。

在技术实现方面，可以结合敏感词库检测、基础 NLP 文本分析以及第三方内容审核 API 实现。

综上所述，在 Quipster 平台中，人工智能技术主要用于信息辅助、情绪识别、内容安全管理以及社交互动增强。通过 AI 问答助手、情绪识别系统、聊天破冰助手以及智能内容识别机制，平台能够在保证系统实现复杂度合理的前提下，为用户提供更加智能化和安全的使用体验。

10 风险分析与未来扩展

在系统开发过程中，可能会面临多种技术与管理风险。为了保证项目能够顺利推进，需要在设计阶段对潜在风险进行分析，并制定相应的缓解措施。同时，在系统初步实现之后，还可以通过持续优化与功能扩展进一步提升平台能力。

10.1 技术风险

在系统开发过程中，可能出现以下技术风险。

系统性能风险

随着用户数量的增加，系统可能面临较高的并发访问压力，例如动态信息流浏览、实时聊天以及推荐计算等功能可能导致服务器负载增加。为降低该风险，系统在架构设计中引入了缓存机制（Redis）以及模块化服务设计，从而提升系统的并发处理能力。

实时通信稳定性风险

即时聊天功能依赖 WebSocket 等实时通信技术。如果连接管理不当，可能会出现消息延迟或连接中断等问题。为解决这一问题，系统将采用成熟的实时通信框架（如 Socket.io），并通过服务器会话管理机制提高通信稳定性。

人工智能模块集成风险

部分功能（如情绪识别与 AI 问答）依赖外部人工智能服务接口。如果接口调用不稳定或成本过高，可能影响系统稳定性。为降低该风险，系统在设计上将 AI 功能作为辅助模块，并保留基于规则或关键词匹配的基础实现方案。

10.2 项目管理风险

在团队协作过程中，还可能出现以下管理风险。

开发进度风险

如果部分功能开发难度高于预期，可能会影响项目整体进度。为减少进度风险，团队采用敏捷开发方法，将开发任务划分为多个迭代周期，并在每个阶段进行阶段性评估与调整。

协作与代码管理风险

多人协作开发可能导致代码冲突或版本管理问题。为解决该问题，项目将使用 Git 进行版本控制，并通过 GitHub 进行代码托管与协作管理。

10.3 未来功能扩展

在系统初步实现之后，平台仍具有较大的扩展空间。未来可能的功能扩展包括：

- 移动端应用开发在 Web 平台基础上开发 iOS 或 Android 移动应用，以提升用户访问便利性。
- 推荐算法优化通过引入更复杂的推荐算法（例如协同过滤或机器学习方法），进一步提升好友匹配与学习搭子推荐的准确性。
- 数据分析与用户画像通过分析用户行为数据，构建用户兴趣画像，从而提供更加个性化的内容推荐。
- 校园服务整合未来可以与校园信息系统进行集成，例如课程信息、校园活动系统或社团管理平台，从而构建更加完整的校园社交生态。

通过对潜在风险的识别与合理规划，Quipster 平台能够在保证系统稳定运行的同时，为未来功能扩展和系统升级提供良好的基础。

11 项目日程详细安排

本项目计划开发周期为 14 周，并采用敏捷开发方法进行项目管理。开发过程划分为多个迭代周期，每个迭代周期约为两周。在每个迭代周期内完成需求实现、功能开发以及阶段性测试，从而逐步完善系统功能。

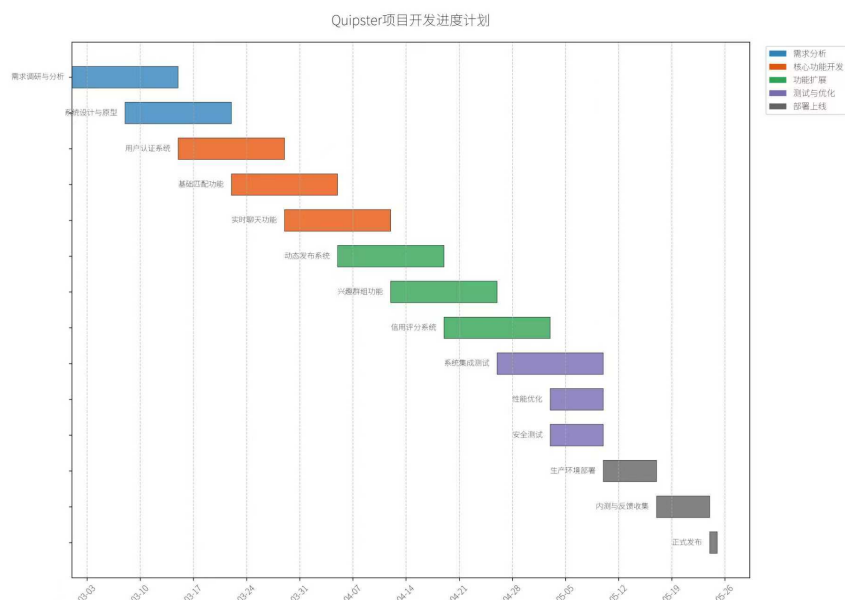


Figure 3: 项目甘特图

References

- [1] S. Shehmir and R. Kashef, "LLM4Rec: A Comprehensive Survey on the Integration of Large Language Models in Recommender Systems—Approaches, Applications and Challenges," *Future Internet*, vol. 17, no. 6, p. 252, 2025. Available: <https://doi.org/10.3390/fi17060252>
- [2] R. Lai, L. Chen, R. Chen, and C. Zhang, "A Survey on Data-Centric Recommender Systems," *arXiv preprint arXiv:2401.17878*, 2024. Available: <https://arxiv.org/abs/2401.17878>
- [3] Y. Li, K. Liu, R. Satapathy, S. Wang, and E. Cambria, "Recent Developments in Recommender Systems: A Survey," *arXiv preprint arXiv:2306.12680*, 2023. Available: <https://arxiv.org/abs/2306.12680>
- [4] Y. Deldjoo, M. Schedl, and P. Cremonesi, "A Review of Modern Recommender Systems Using Generative Models," *arXiv preprint arXiv:2404.00579*, 2024. Available: <https://arxiv.org/abs/2404.00579>
- [5] S. Raza and M. Ding, "A Comprehensive Review of Recommender Systems: Transitioning from Theory to Practice," *arXiv preprint arXiv:2407.13699*, 2024. Available: <https://arxiv.org/abs/2407.13699>
- [6] X. Chen, L. Yao, J. McAuley, G. Zhou, and X. Wang, "A Survey of Deep Reinforcement Learning in Recommender Systems: A Systematic Review and Future Directions," *ACM Computing Surveys*, 2021. Available: <https://arxiv.org/abs/2109.03540>
- [7] C. Gao, X. Wang, X. He, and Y. Li, "A Survey of Graph Neural Networks for Recommender Systems: Challenges, Methods, and Directions," *IEEE Transactions on Knowledge and Data Engineering*, 2021. Available: <https://arxiv.org/abs/2109.12843>
- [8] H. Lin and Z. Chen, "A Survey on Diffusion Models for Recommender Systems," *arXiv preprint arXiv:2409.05033*, 2024. Available: <https://arxiv.org/abs/2409.05033>
- [9] X. Wang, Y. Chen, and J. Liu, "A Survey on Fairness-Aware Recommender Systems," *Information Fusion*, vol. 100, p. 101906, 2023. Available: <https://doi.org/10.1016/j.inffus.2023.101906>
- [10] P. R. Dasari, "Evolution of Software Testing and the Role of AI and Machine Learning," *International Journal of Computer Engineering and Technology*, vol. 15, no. 3, pp. 242–251, 2024. Available: <https://doi.org/10.34218/IJCET.15.03.023>
- [11] H. V. Pandhare, "Future of Software Test Automation Using AI/ML," *International Journal of Engineering and Computer Science*, vol. 14, no. 5, pp. 27159–27182, 2025. Available: <https://doi.org/10.18535/ijecs.v14i05.5139>

- [12] J. Zhang, "Improving Software Quality Assurance with Automated Testing Frameworks," *Indonesian Journal of Multidisciplinary Innovations*, 2024. Available: <https://scholarlyarticle.vncinstitute.com/index.php/IJMI/article/view/43>